ECE 425-525: Course Project, Final Report

Project ID: P04

VLSI Design Agent

An interactive toolkit for RTL generation, testing, and improvement

To write: Team members' names and roles (agent and integration; EDA and evaluation; dashboard and delivery).

Abstract. We built an agent that writes Verilog RTL and a self-checking testbench from a specification using a local language model (gemma4:26b), checks both with Icarus Verilog and Yosys, and revises from the tool feedback for up to 3 attempts, together with a browser dashboard and a container image that make it usable in a course. Hidden reference tests that the agent never sees score every attempt, so each run yields a paired one-shot and iterative result. Over 200 runs on 10 circuits with four prompt strategies, revising fixed 10 one-shot failures and broke 4 correct designs; most fixes were compile errors that the tools named by line, and no revision fixed a specification-level logic bug unless the testbench was written separately from the design. The bottleneck is the model's own testbench: on circuits held out during development it rejected 54–81% of the designs that were in fact correct, so most revisions went into designs that were already right.

1. Introduction

Large language models can write Verilog that looks right and compiles, yet describes the wrong circuit. That gap is the point of departure of our proposal: a student should be able to watch an agent generate a design, see a test fail, inspect the revision, and learn which settings actually make the hardware correct. Tool feedback seems the obvious remedy. A compiler names the line it rejects and a testbench names the input it fails, so an agent that feeds these messages back to the model should improve on its first attempt.

We test that claim directly. Our agent generates a design and a testbench from a specification, runs development checks (compilation, simulation of its own testbench, synthesis), and revises from their results. A separate reference test, written by the team and never shown to the agent, decides whether a design is correct. We ask two questions: *does iterating on tool feedback beat one-shot generation*, and *how reliable is the feedback that the agent gives itself*?

This report describes:

- the agent, with a single-conversation workflow and a workflow that writes the testbench from the specification before any design exists (Section 3);
- an evaluation that keeps the agent away from the tests that score it, scores every attempt so that one run gives paired one-shot and iterative results, and uses a held-out set of circuits declared in advance (Sections 3 and 5);
- the toolkit students use: a dashboard to run the agent, inspect every attempt down to waveforms, edit and re-test its RTL, compare strategies and write new ones, delivered as a tested, published container image (Section 4);
- the results of 200 benchmark runs and what they say about where agentic RTL generation fails (Sections 6 and 7).

2. Related work

Benchmarks. Thakur et al. [1] fine-tuned open code models on Verilog collected from GitHub and textbooks and tested them with testbenches on 17 problems of graded difficulty; the fine-tuned CodeGen-16B outperformed the commercial Codex model on functional correctness. VerilogEval [2] turned 156 problems from the HDLBits exercise site into text-only descriptions, one set written by hand and one generated with GPT-3.5, and scores a completion by simulating it with Icarus Verilog against a golden solution, reporting pass@k. Its revision [3] adds specification-to-RTL tasks and classifies failures automatically. RTLLM [4] provides 30 designs, each with a description, a testbench and a hand-written reference design, and grades three goals in turn: syntax, functionality, and design quality as power, performance and area after synthesis. In all of them the test that decides correctness is written by the benchmark's authors, never by the model under test. Our hidden reference tests follow the same rule.

Loops with feedback. In Chip-Chat [5] an experienced engineer worked with ChatGPT-4 to design an 8-bit accumulator-based processor that went to tapeout on a SkyWater 130 nm shuttle; the engineer steered the conversation and verified the output, writing the testbenches. AutoChip [6] replaced the engineer with tools: each candidate is compiled with Icarus Verilog and simulated with a testbench supplied with the problem, and the errors and mismatches go back to the model until every test passes or an iteration limit is reached. With feedback from only the latest iteration it produced 24.2% more functionally correct code than zero-shot generation. RTLFixer [7] found about 55% of the errors in LLM-written Verilog to be syntax errors and, with retrieval-augmented generation and ReAct prompting, fixed about 98.5% of the compilation errors in its dataset. Multi-agent systems go further. VerilogCoder [8] plans a module as a dependency graph of sub-tasks and gives its debug agent a simulator and a waveform tracer that follows a mismatched signal back through the abstract syntax tree. MAGE [9] samples many candidate designs at high temperature, scores them in simulation, and debugs them with checkpoints of the expected internal state that locate the first mismatch. With GPT-4 Turbo and Claude 3.5 Sonnet respectively, they report 94.2% and 95.7% on the revised VerilogEval [3].

Who writes the test. These loops differ in where the test that drives them comes from, and that is the question our study isolates. In AutoChip the testbench that produces the feedback is also the one that decides success, and VerilogCoder's debug agent revises until the code passes a provided testbench. MAGE has an agent write the testbench from the specification, combined with the golden testbench when one is available, and a judge agent that regenerates the testbench when it decides that the testbench, not the design, is wrong. Writing the test is itself hard: in AutoChip's evaluation the problems that asked for a testbench fared worst, with none of the generated code compiling, and AutoBench [10], an LLM testbench generator, reports that generating testbenches directly has a low pass rate. AutoBench counts a generated testbench as preliminarily correct only if it reports no errors on the golden design, the same false-rejection measure we apply to our agent's testbenches (Section 6.1).

Self-correction. Self-Refine [11] has one model write, critique and revise its own output, and reports an average gain of about 20% absolute over one-step generation across seven tasks. Huang et al. [12] find that without external feedback, models struggle to correct their own reasoning and sometimes get worse. Our agent sits between the two: the compiler and the synthesizer give external feedback, but the testbench that judges behavior is the model's own, so the feedback on function is only as good as the model's reading of the specification.

Adapted models. Another line of work changes the model rather than the loop. RTLCoder [13] fine-tunes a 7B model on more than 27,000 instruction–code pairs generated with GPT-3.5 and outperforms GPT-3.5 on RTL benchmarks, arguing that a local open model removes the privacy concern of uploading every instruction to a commercial provider. ChipNeMo [14] adapts LLaMA2 models to chip design through domain-adaptive tokenization, continued pretraining, alignment and a domain-

adapted retrieval model, and evaluates them as an engineering assistant chatbot, for EDA script generation and for bug summarization. We used an unmodified model through Ollama, so our results describe an off-the-shelf model inside the loop; fine-tuning was a stretch goal we did not attempt (Section 11).

What this study adds. We do not propose a stronger generator: our circuits are smaller than these benchmarks' and our model (gemma4:26b) runs locally. What we add is measurement. First, the agent never sees the test that scores it, and every attempt is scored, so each run yields a paired one-shot and final result and counts both the failures that revision fixed and the correct designs it broke. Second, we measure the agent's own checks against that independent test, which shows how often a model-written testbench rejects a correct design: the quantity that decides whether a loop like AutoChip's can work where no reference testbench exists. Third, the held-out circuits and runs were declared before each round, and strategies were tuned on the dev circuits only.

3. System design

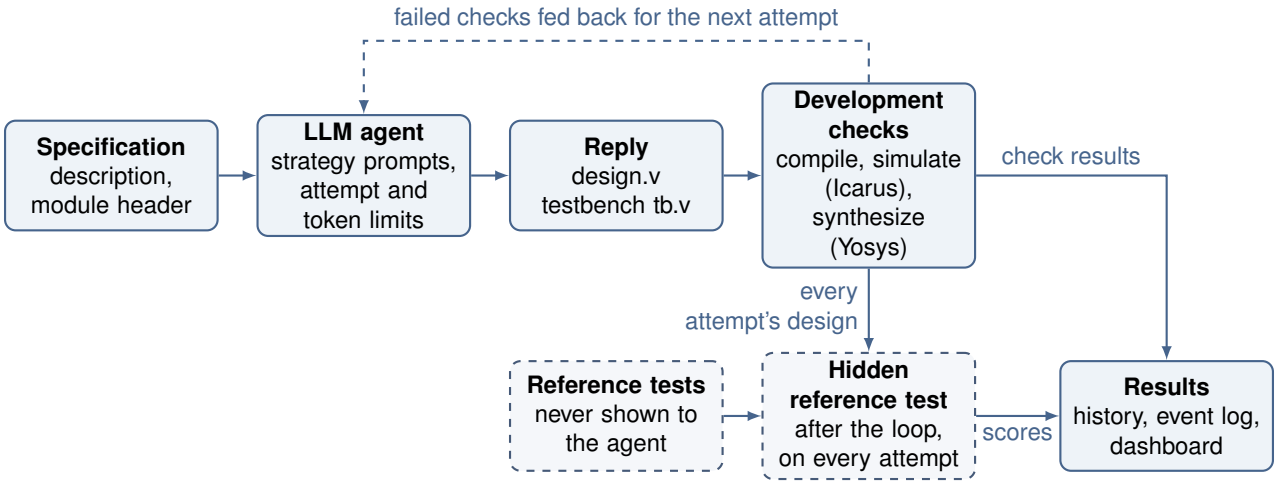


Figure 1: The agent loop. Only the development checks reach the model; the hidden reference test scores the designs after the loop ends.

Figure 1 shows the loop. The model receives the circuit's description and its exact module header and replies with two Verilog blocks: the design and a self-checking testbench that must end by printing `RESULT: passed/total passed`. The framework extracts the code, compiles design and testbench with Icarus Verilog, runs the testbench, and synthesizes the design with Yosys, flagging inferred latches. If any of these development checks fails, their output (capped at a few thousand characters) goes back to the model with a request to fix the design or, if a check itself is wrong, the testbench. The run stops when the checks pass, after 3 attempts, or when its budget of 50,000 tokens is spent. Every tool runs with a timeout and capped output, so a design that hangs the simulator ends as a timeout instead of stopping the agent.

The second workflow, `spec_first`, splits the work into two conversations. One writes the testbench from the specification alone, before any design exists, and must show that it runs against a placeholder design. The other writes the design and never sees the testbench. When a design fails some of the testbench's checks, the failing checks first go back to the testbench's author, who either corrects its testbench or confirms it; only then does the design conversation get the feedback.

Every run writes an event log as it happens and a history file at the end with the settings, every model call, every attempt's checks and scores, and a copy of the prompts the run used, so that results stay interpretable after a strategy is edited.

3.1 Keeping the evaluation honest

The agent is scored only by the hidden reference tests. They are written by the team, kept outside everything the model sees, and run only after the loop has finished. A generated design could in principle print a fake result line and stop the simulation; to rule that out, each run replaces a placeholder in the reference test's result line with a fresh random token, and only a line carrying that token counts. The reference tests are exhaustive for the combinational circuits and combine directed and random sequences for the sequential ones.

Because every attempt is scored, attempt 1 is exactly what a one-shot run would have produced, and the final attempt is the iterative result. Each run therefore gives a paired comparison, and shows when a revision fixed a failing design or broke a correct one.

The reference tests are themselves tested. Each circuit comes with hand-written correct and deliberately faulty designs (58 in all, 20 of them correct). Every correct design must pass, and every faulty one must fail for its intended reason: a dropped carry, a reset of the wrong kind, swapped multiplexer selects, a signed comparison done unsigned, a simulation that never finishes. The acceptance checks in `docs/acceptance-checks.md` record this together with each team member's completion check.

3.2 Circuits

Circuit	Family	Split	Hidden-test checks	Fixtures correct / faulty
`adder4`	adder	Dev	512	2 / 3
`mux2`	mux	Dev	524	2 / 4
`counter4`	counter	Dev	249	2 / 4
`alu4`	alu	Dev	2,048	2 / 5
`seqdet_moore`	fsm	Dev	686	2 / 4
`adder8`	adder	Held-out	131,072	2 / 3
`mux4`	mux	Held-out	1,064	2 / 3
`updown8`	counter	Held-out	380	2 / 4
`alu8`	alu	Held-out	524,288	2 / 4
`seqdet_mealy`	fsm	Held-out	684	2 / 4

Table 1: The 10 circuits. Strategies were tuned on the dev circuits only; the held-out circuits were used once per strategy, with the runs declared in advance.

The circuits come in pairs of the same family (Table 1), one for development and one held out: adders, multiplexers, counters, ALUs with eight operations and four flags, and overlapping sequence detectors with Moore and Mealy outputs. The first six circuits proved too easy, as the model passed nearly all of them in one shot, so the ALU and sequence-detector pairs were added before any benchmark run to leave room for iteration to matter.

4. The toolkit

The dashboard is a browser application (Gradio and Plotly) organized in five sections. A student picks a circuit, whose specification and module header are shown, a strategy, one-shot or iterative mode, the attempt limit and the model, and watches the run's log stream in as it happens (Figure 2). A scripted model replays a recorded adder4 run, so the interface can be tried without a GPU.

The results section opens any saved run, whether started from the dashboard, the command line or a benchmark, and shows for each attempt the agent's checks next to the hidden test's verdict, the design and testbench, the changes from the previous attempt, the hidden test's mismatch lines, the

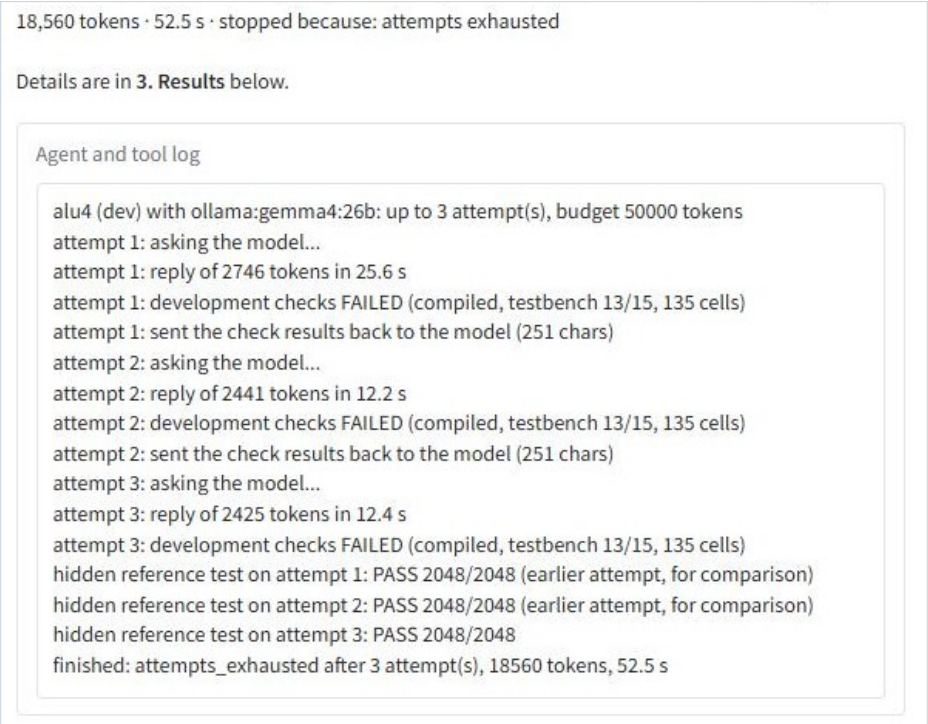


Figure 2: A run of the agent on alu4: the dashboard's status line and live log. The model's own testbench fails every attempt (13/15 checks), so the agent revises twice and stops at the limit, while the hidden test passes all three designs.

feedback the model received, and the model's raw reply. A waveform view re-runs the hidden test on any attempt and plots the circuit's inputs and outputs, with the test's expected values beneath them, around the first mismatch (Figure 3).

Students can also edit an attempt's design or testbench and run exactly the checks the agent's attempts get. The edited code goes through the same loop as a single attempt, and the result is saved as a new run that records where it came from (Figure 4). The remaining sections chart any benchmark (Figure 5) and provide a strategy editor, which copies a strategy, checks its template placeholders, previews the prompts as the model will receive them, and saves the copy without touching the shipped strategies. Runs and benchmark results can be exported as archives and standalone pages.

For delivery, the toolkit ships as a container image with the exact tool versions the benchmarks used: Icarus Verilog 12, Yosys built from the same commit, and pinned Python packages. A Compose file starts the dashboard together with an Ollama server on the GPU and downloads the model on first start; runs and custom strategies are kept in the project folder across container restarts. A continuous-integration workflow builds the image on every push to the main branch, runs the fixture check and the test suite inside it, and only then publishes it, so a teammate can pull a tested image instead of building one. Following Member 3's completion check, a fresh clone with the published image was used to open the dashboard, run an example, edit and rerun a strategy, and export the result, and the runs and strategy survived recreating the container.

5. Experimental method

All runs used gemma4:26b through Ollama on one RTX 4090, at temperature 0.2 with thinking disabled, at most 4,096 output tokens per reply, and up to 3 attempts and 50,000 tokens per run (no run reached the budget). Each strategy ran every circuit with seeds 1–5. With the same seed and prompt a run repeats exactly, so each seed is one independent sample.

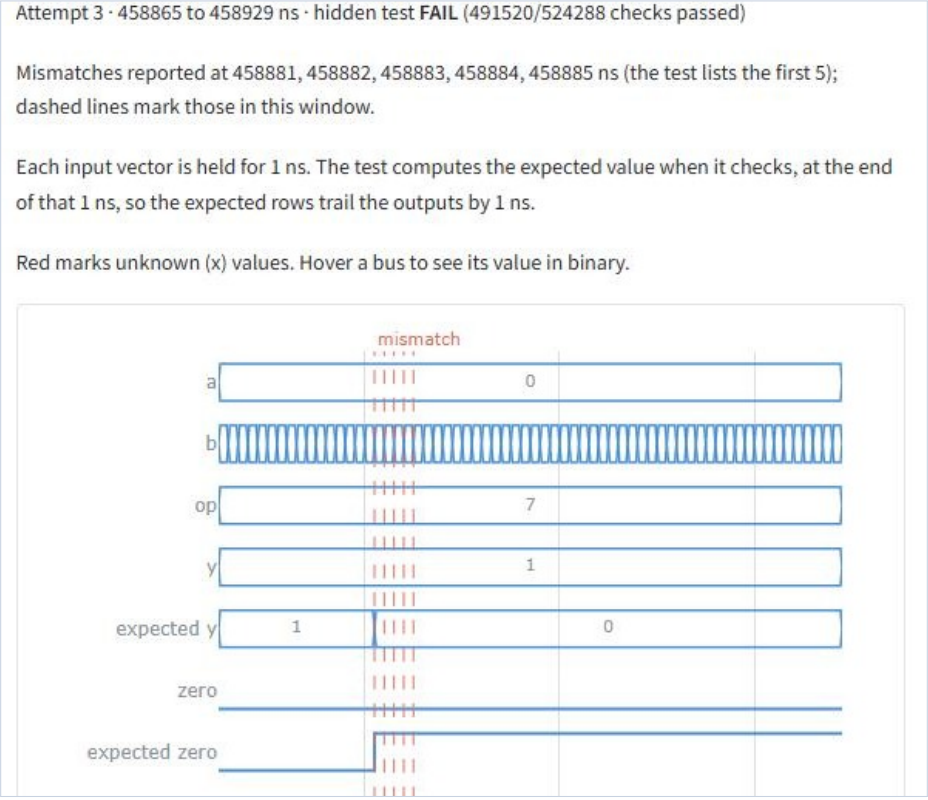


Figure 3: The waveform view on a held-out alu8 design with a signed-comparison bug. For the operation SLT (op 7) the design keeps `y = 1` while the expected value drops to 0 once `b` becomes negative; dashed lines mark the reported mismatches.

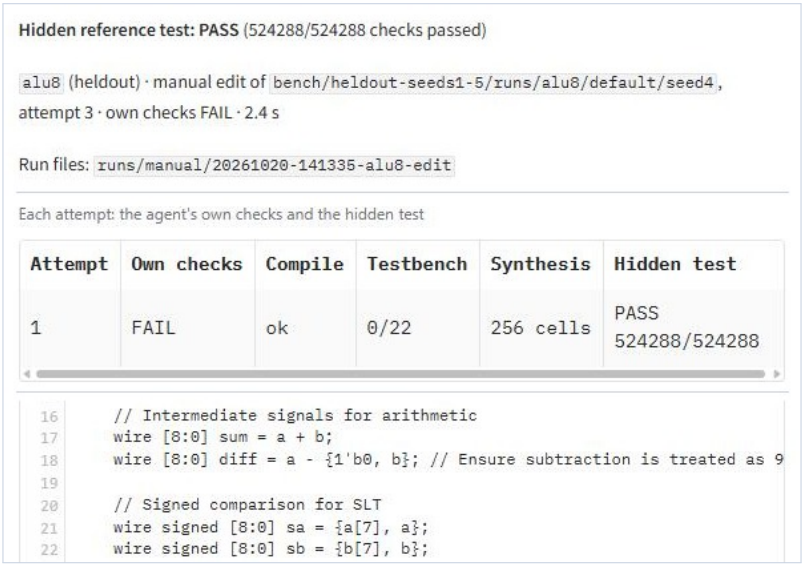


Figure 4: Editing and re-testing the design of Figure 3. Declaring the two sign-extended operands `signed` makes the hidden test pass all 524,288 checks, while the model's own testbench still reports 0/22.



Figure 5: Two of the dashboard's five benchmark charts for the held-out circuits: passes after each attempt, and passes per circuit (one-shot → iterative). The dashboard names strategies by folder, so `spec_first` there is version 2.

We compared four strategies:

- `default`: the starter prompts, kept unchanged as the baseline;
- `tb_timing`: the same, plus advice on when a testbench should sample a clocked output;
- `spec_first` v1: the two-conversation workflow of Section 3;
- `spec_first` v2: the same, but the testbench's author must quote the sentence of the specification behind any expected value it changes.

We report, as the proposal promised, compilation, hidden-test pass rate, synthesis, attempts used, time and tokens, and cell counts only for designs that pass the hidden test, all synthesized with the same generic Yosys flow. The work proceeded in a fixed order: a difficulty check with one-shot runs on the four harder circuits (seeds 101–105), then strategies were developed and compared on the dev circuits only, declared in a commit, and run once on the held-out circuits without further tuning. Round 1 compared `default` and `tb_timing`; round 2 added both versions of `spec_first`.

6. Results

Strategy	One-shot	Iterative	Fixed	Broken	Pass after 1 / 2 / 3	Own checks reject correct	Mean tokens	Mean time
Held-out circuits								
default	22/25	24/25	2	0	22 / 24 / 24	81%	10,501	23.7 s
tb_timing	25/25	25/25	0	0	25 / 25 / 25	54%	6,780	18.3 s
spec_first v1	20/25	21/25	2	1	20 / 21 / 21	65%	12,526	26.1 s
spec_first v2	20/25	23/25	3	0	20 / 23 / 23	59%	12,817	25.9 s
Dev circuits								
default	24/25	24/25	0	0	24 / 24 / 24	51%	7,296	16.6 s
tb_timing	23/25	22/25	0	1	23 / 22 / 22	36%	5,906	15.9 s
spec_first v1	23/25	24/25	1	0	23 / 24 / 24	21%	5,985	15.4 s
spec_first v2	23/25	23/25	2	2	23 / 24 / 23	32%	6,744	15.6 s

Table 2: One-shot vs. iterative, per strategy and split (25 runs each). Green: revising helped overall; red: it hurt. "Own checks reject correct": share of correct designs (by the hidden test) that the agent's own development checks failed.

Revising from the agent's own feedback rarely changed the outcome (Table 2). Over all 200 paired runs, revisions fixed 10 designs and broke 4. On the held-out circuits `default` went from 22/25 to 24/25, and both of its fixes were the same compile error on mux4; `tb_timing` passed every held-out run in one shot. The `spec_first` versions started lower on held-out (20/25 and 20/25) and recovered to 21/25 and 23/25. Figure 6 shows the same results as if every run had stopped after one, two or three attempts: nearly every change happens at attempt 2, and the third attempt never added a pass (for `spec_first` v2 on dev it lost one).

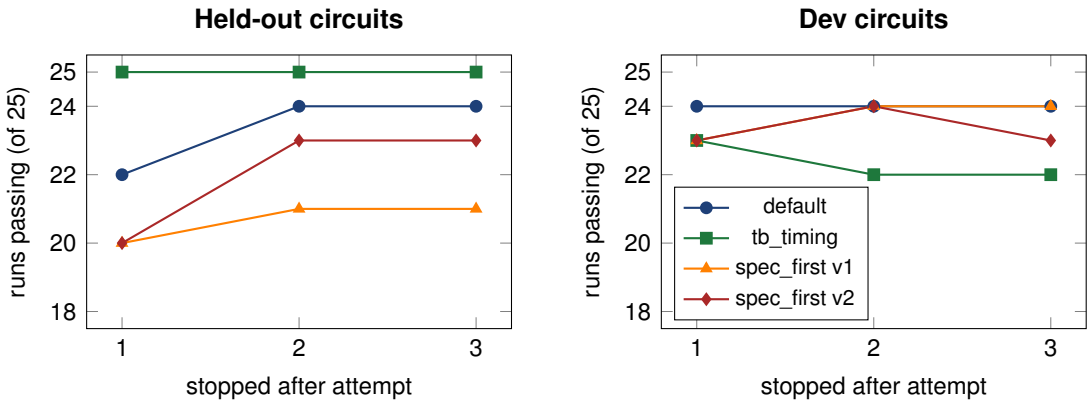


Figure 6: Hidden-test passes if each run had stopped after 1, 2 or 3 attempts (attempt 1 is the one-shot result). The vertical axis starts at 17.5.

The strategies' differences are small and do not hold across splits. Combined over dev and held-out circuits the four strategies scored 45–48/50, and their order flips: `tb_timing` was the worst on dev and the best on held-out. With 25 runs per cell, differences of one to three runs are within noise; for example, 22/25 against 25/25 gives $p \approx 0.23$ in Fisher's exact test.

Table 3 keeps apart the outcomes the proposal named: compilation, synthesis and the hidden test. In all 200 runs every design that compiled also synthesized, so the first two columns agree. Every one-shot compile failure was on a held-out circuit, and there revising raised the number of compiling designs for three of the four strategies.

Strategy	Compiled	Synthesized	Passed the hidden test	Mean attempts
Held-out circuits				
default	23 → 25	23 → 25	22 → 24	2.5
tb_timing	25 → 25	25 → 25	25 → 25	1.6
spec_first v1	22 → 23	22 → 23	20 → 21	1.9
spec_first v2	21 → 24	21 → 24	20 → 23	1.8
Dev circuits				
default	25 → 25	25 → 25	24 → 24	1.7
tb_timing	25 → 25	25 → 25	23 → 22	1.6
spec_first v1	25 → 25	25 → 25	23 → 24	1.2
spec_first v2	25 → 24	25 → 24	23 → 23	1.4

Table 3: Compilation, synthesis and hidden-test outcomes, one-shot → final, out of 25 runs per strategy and split, and the mean number of attempts per run.

Tables 4 and 5 locate the changes. The adders never failed. Apart from mux4's compile errors and revisions that broke two counters (updown8 and counter4), every failure was on an ALU or a sequence detector. The median cell counts of passing designs are listed for completeness; as the proposal noted, they are a rough area measure, not power, area or timing.

Circuit	default	tb_timing	spec_first v1	spec_first v2	Median cells
`adder8`	5	5	5	5	52
`mux4`	3 → 5	5	5	5	15
`updown8`	5	5	5 → 4	5	47
`alu8`	4	5	4	4	256
`seqdet_mealy`	5	5	1 → 3	1 → 4	9

Table 4: Held-out circuits: passing runs out of 5, one-shot → iterative, and the median cell count of passing final designs.

Circuit	default	tb_timing	spec_first v1	spec_first v2	Median cells
`adder4`	5	5	5	5	23
`mux2`	5	5	5	5	8
`counter4`	5	5	5	5 → 3	10
`alu4`	4	4 → 3	3 → 4	3 → 5	134
`seqdet_moore`	5	4	5	5	18

Table 5: Dev circuits, as in Table 4.

6.1 How reliable were the agent's own checks?

Every attempt that produced code was judged twice: by the agent's development checks and by the hidden test. Figure 7 and Table 6 compare the two. On the held-out circuits every strategy's checks rejected 54–81% of the designs that were in fact correct (21–51% on dev), while they accepted few wrong designs. A correct design that fails the agent's checks gets revised anyway, which is where most of the extra cost goes: the 114 runs that stopped after one attempt used 2,700 tokens on average, the 59 that used all three attempts 20,085.

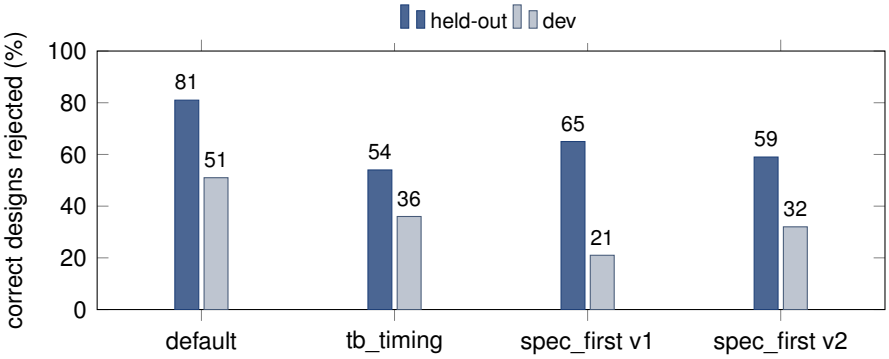


Figure 7: Share of correct designs (passing the hidden test) that the agent's own development checks rejected, over all attempts.

	Correct designs rejected		Wrong designs accepted	
Strategy	count	share	count	share
Held-out circuits				
default	47 of 58	81%	0 of 5	0%
tb_timing	22 of 41	54%	none	–
spec_first v1	24 of 37	65%	2 of 11	18%
spec_first v2	22 of 37	59%	1 of 8	12%
Dev circuits				
default	20 of 39	51%	0 of 3	0%
tb_timing	12 of 33	36%	2 of 6	33%
spec_first v1	6 of 29	21%	1 of 2	50%
spec_first v2	10 of 31	32%	1 of 5	20%

Table 6: The agent's own checks against the hidden test, over all scored attempts.

In the `spec_first` runs, a failing design first sent the testbench's author back to review its checks. Table 7 judges each review by whether the design under review actually passed the hidden test. Version 1's author revised its testbench in 30 of 40 reviews and gave in to a buggy design 5 times; version 2's author, required to quote the specification, revised in only 8 of 40 but kept a testbench that was rejecting a correct design 28 times. Neither judged its own testbench reliably.

		Right call		Wrong call	
Version	Reviews	revised, design correct	kept, design wrong	kept, design correct	revised, design wrong
Held-out circuits					
spec_first v1	28	15	1	9	3
spec_first v2	26	6	1	18	1
Dev circuits					
spec_first v1	12	10	0	0	2
spec_first v2	14	1	3	10	0

Table 7: `spec_first`: testbench reviews after a design failed some checks, judged by the hidden test of the design under review.

7. Discussion

What iteration fixes. The revisions that turned failures into passes were mostly compile errors that the tools reported by line. In both held-out mux4 fixes the design assigned the output inside an `always` block but declared it as a plain output; Icarus Verilog reported `y is not a valid l-value`, and the model changed one declaration. Most of `spec_first`'s held-out recoveries on seqdet_mealy were the same kind of error.

What it does not fix. No revision in round 1 fixed a specification-level logic bug. The dominant one was the ALUs' set-less-than: the specification asks for a signed comparison, and the model sign-extended both operands and compared them with <, which remains unsigned (Appendix A). The feedback could not help, because it came from the model's own testbench, and that testbench was wrong in other ways.

Why: the checks are the bottleneck. The proposal named as its biggest risk that the model would write a design and a testbench with the same mistake. That happened, but the larger problem was broader: the model's testbenches sampled clocked outputs at the wrong moment, computed wrong expected values, failed to compile, or never printed a result. A loop is only as good as its checks, and here the checks rejected most correct designs, so most revisions were spent on designs that were already right, and occasionally broke them.

Separating the testbench's author helps only partly. Writing the testbench first, in its own conversation, lowered false rejections on the dev circuits from 51% to 21% and was the only strategy whose revisions fixed the signed-comparison bug (3 dev runs). On the held-out circuits the gain did not hold: false rejections stayed at 65% and 59%, and the strategy ended at 21/25 and 23/25 against 24/25 and 25/25 for the simpler strategies, at the highest token cost. Requiring the author to quote the specification stopped it from giving in to buggy designs, but it then defended testbenches whose expected values were right and whose timing was wrong.

For a teaching toolkit this points to where effort pays off: better checks (for example a testbench harness that fixes when outputs are sampled) rather than more elaborate revision prompts. The dashboard's edit-and-re-test view makes the point tangible: a one-word fix passes the hidden test while the model's own testbench still fails it.

8. Limitations and threats to validity

- **Small samples.** Each strategy has 25 runs per split; differences of one to three runs are within noise.
- **One dominant bug.** Most logic failures were the ALUs' signed comparison, so conclusions about which logic bugs iteration can fix rest mostly on one bug type.
- **One model and setting.** All runs used gemma4:26b at temperature 0.2; other models or settings may revise differently.
- **Small circuits.** At most 8 bits and a handful of states.
- **Strategies change the first prompt too.** `tb_timing`'s and `spec_first`'s one-shot results differ from `default`'s partly because their prompts differ, so the comparison is not a pure test of feedback.
- **Bounded evidence.** Passing a reference test is strong evidence, not proof; the reference tests were written by the team and checked with fixtures. Cell counts are not power, area or timing.
- **Held-out circuits used more than once.** They served in the difficulty check and in both rounds, though strategies were tuned on dev only and each round's held-out runs were declared beforehand.

- **Stretch goals not attempted.** Retrieval-augmented generation and LoRA fine-tuning were left out.

9. Reproducibility

Everything behind this report is public.

- **Repository:** `https://github.com/Harrydhara/VLSI-Design-Agentic-Framework`, with every benchmark run under `runs/bench/`.
- **Container image:** `ghcr.io/harrydhara/vlsi-design-agentic-framework`, which pins Icarus Verilog 12, Yosys 0.57+260 built from the benchmark commit, Ollama 0.31.2 and the Python packages.
- **Commands:** `vlsi-agent bench` reruns a benchmark and resumes where it stopped; `docs/final-report/generate.py` regenerates this report's tables and figures from the runs, and `docs/report/build.py` the longer benchmark report. Both use the same summary code, and a test checks that their numbers agree with the dashboard's.

Repeating a run with the same seed and prompt reproduces it exactly: an alu4 run started from the dashboard matched an earlier command-line run token for token (18,560 tokens, the same three verdicts).

10. Deliverables against the proposal

Table 8 compares what the proposal promised, and the completion checks of the division of work, with what was delivered. Everything in the minimum viable product and the final submission was delivered, with two gaps: the dashboard has a student choose one of the ten circuits rather than type in a new specification, and the short introductions to logic gates, Verilog and simulation for students from other majors were not written. Of the stretch goals, the 4-bit ALU was delivered, while retrieval-augmented generation and LoRA fine-tuning were not attempted.

To write: Say in the team's words how the stack differs from the proposal: it named a hosted LLM API or PyTorch/Hugging Face Transformers for inference; we used a local model through Ollama, which kept runs free, private and reproducible on one GPU.

11. Conclusion and future work

Iterating on tool feedback improved one-shot generation only slightly: it reliably repaired errors the tools could name, but rarely the logic, because the model's own tests rejected most correct designs and missed or shared the real bugs. Making an agent better at hardware therefore starts with making its checks trustworthy.

Next steps follow from that: a testbench harness that provides the clock, reset and sampling time, so the model writes only stimulus and expected values; keeping the last design that compiled and stopping when a revision repeats the previous design; retrieval of verified Verilog examples and fine-tuning with separate training and evaluation data, the stretch goals left open; a review agent that judges disputed checks; and larger circuits such as a FIFO, more seeds and a second model.

Promise	Status	Evidence
MVP: generate, simulate, revise and synthesize adders, multiplexers and counters	Done	Sections 3–4
Compare baseline and improved prompts on held-out specifications with independent reference tests	Done	200 paired runs, Section 6
Dashboard: choose a circuit and see its specification, edit prompts and RTL, one-shot or iterative, attempt limit, run, test, synthesize, reset, export	Done	Figures 2–5
Dashboard: enter a new circuit description and bit width	Partly	Ten circuits in five families to choose from; a new specification is added as files with its own reference test (README)
Outputs: Verilog, revision diffs, waveforms, pass/fail, cell counts, tool logs	Done	Section 4
Metrics: compilation, test pass rate, synthesis, retries, latency, tokens; cells only for passing designs	Done	Tables 2–5
Controlled tool environment (Docker) and Git	Done	Image, Compose, published by CI
Stretch: 4-bit ALU	Done	alu4 and alu8, plus two sequence detectors
Stretch: RAG; LoRA fine-tuning	Not done	Section 11
Optional FPGA board	Not used	–
Guided examples and short introductions for students from other majors	Partly	A scripted example run and documented strategy examples; no introductions to logic gates, Verilog or simulation
Final deliverables: report, documented repository, live demonstration	Done	This report; README; the presentation's demo
Completion checks of all three team members (division of work)	Pass	`docs/acceptance-checks.md`

Table 8: The proposal's promises and the division of work's completion checks.

References

[1] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking large language models for automated Verilog RTL code generation," in *Proc. Design, Automation & Test in Europe Conf. (DATE)*, 2023, pp. 1–6, doi:10.23919/DATE56975.2023.10137086.

[2] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. IEEE/ACM Int. Conf. on Computer Aided Design (ICCAD)*, 2023, pp. 1–8, doi:10.1109/ICCAD57390.2023.10323812.

[3] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany, "Revisiting VerilogEval: A year of improvements in large-language models for hardware code generation," *ACM Trans. Design Automation of Electronic Systems*, vol. 30, no. 6, pp. 1–20, 2025, doi:10.1145/3718088.

[4] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An open-source benchmark for design RTL generation with large language model," in *Proc. Asia and South Pacific Design Automation Conf. (ASP-DAC)*, 2024, pp. 722–727, doi:10.1109/ASP-DAC58780.2024.10473904.

[5] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2023, pp. 1–6, doi:10.1109/MLCAD58807.2023.10299874.

[6] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, “AutoChip: Automating HDL generation using LLM feedback,” arXiv:2311.04887v2, 2024.

[7] Y.-D. Tsai, M. Liu, and H. Ren, “RTLFixer: Automatically fixing RTL syntax errors with large language model,” in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, 2024, pp. 1–6, doi:10.1145/3649329.3657353.

[8] C.-T. Ho, H. Ren, and B. Khailany, “VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool,” in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 39, no. 1, 2025, pp. 300–307, doi:10.1609/aaai.v39i1.32007.

[9] Y. Zhao, H. Zhang, H. Huang, Z. Yu, and J. Zhao, “MAGE: A multi-agent engine for automated RTL code generation,” in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, 2025, pp. 1–7, doi:10.1109/DAC63849.2025.11133191.

[10] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li, “AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design,” in *Proc. ACM/IEEE Int. Symp. on Machine Learning for CAD (MLCAD)*, 2024, pp. 1–10, doi:10.1145/3670474.3685956.

[11] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-Refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[12] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

[13] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, “RTLCoder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution,” in *Proc. IEEE LLM Aided Design Workshop (LAD)*, 2024, pp. 1–5, doi:10.1109/LAD62341.2024.10691788.

[14] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, et al., “ChipNeMo: Domain-adapted LLMs for chip design,” arXiv:2311.00176, 2023.

A. A worked example: the signed-comparison bug

Run `runs/bench/heldout-seeds1-5/runs/alu8/default/seed4`. The specification defines operation 111:

```
111  SLT   1 if a < b as signed two's complement numbers, otherwise 0
```

All three attempts implemented it the same way:

```
wire [8:0] sa = {a[7], a};
wire [8:0] sb = {b[7], b};
...
3'b111: begin // SLT
    y_reg = (sa < sb) ? 8'h01 : 8'h00;
end
```

Sign extension does not make < signed: both vectors are still unsigned, so $b = -128$ becomes `9'h180` and compares as larger than 0. The model’s own testbench could not reveal this. It failed every one of its checks with all outputs unknown. This is how the feedback after attempt 1 began (after attempt 2 it was the same with 0/22):

```
- The testbench reported failures (0/23 checks passed):
FAIL: a=170 b= 85 op=000 | Expected: y=  0 z=1 c=0 v=0 n=0 | Actual: y=  x z=x c=x v=x n=x
FAIL: a=255 b= 15 op=000 | Expected: y= 15 z=0 c=0 v=0 n=0 | Actual: y=  x z=x c=x v=x n=x
...
```

The hidden test scored all three attempts 491,520 of 524,288 and pinpointed the operation:

```
MISMATCH op=111 a=0 b=128: expected y=0 zero=1 carry=0 overflow=0 negative=0, got y=1 zero=0 ...
```

Figure 3 shows the same failure as a waveform. Editing the design in the dashboard to declare `wire signed [8:0] sa` and `sb` makes the hidden test pass all 524,288 checks, while the model's testbench still reports 0/22 (Figure 4; the run is saved in `runs/manual/`).

B. The baseline prompts

The `default` strategy, unchanged throughout the project.

system.txt

```
You are a careful digital design engineer. You write synthesizable Verilog-2005 RTL and a self-checking
↪  testbench for it.

Rules for the design:
- Use exactly the module header you are given: the same module name, port names, directions and widths.
- Write synthesizable RTL only: no # delays, no initial blocks, and no $display or other system tasks in the
↪  design.
- Combinational logic: use assign, or always @(*) that assigns every output in every branch, so no latches are
↪  inferred.
- Sequential logic: use always @(posedge clk ...) with nonblocking assignments (<=).
- Helper modules are allowed; put them in the same code block as the design.

Rules for the testbench:
- Name the testbench module exactly tb, with no ports, and instantiate the design inside it.
- Apply inputs and compare every output with the value the specification requires. Cover corner cases.
- If the design has a clock, generate it with: always #5 clk = ~clk;
- Count the checks. For every failed check, print a line that starts with FAIL: and shows the inputs, the
↪  expected value and the actual value.
- At the end, print exactly one summary line and stop:
    $display("RESULT: %0d/%0d passed", passed, total);
    $finish;

Reply format: exactly two fenced code blocks and nothing else that is code. First a ```verilog block with the
↪  design, then a ```verilog block with the testbench.
```

task.txt

```
Design the following circuit.

{{description}}

Use exactly this module header:
{{interface}}

Write the design (top module {{top}}) and a self-checking testbench module named tb.
```

feedback.txt

```
Attempt {{attempt}} of {{max_attempts}} did not pass the development checks:

{{feedback}}

Fix the problems. If a check failed because the testbench itself is wrong, fix the testbench; otherwise fix the
↪  design. Reply with both complete code blocks again: the design first, then the testbench.
```